Introduction to NLP

AIAA 4051 Lecture 24

Instructor: Sihong Xie

Email: sihongxie@hkust-gz.edu.cn

AI Thrust, HKUST(GZ)

Score-Based Perspective: The Score Function

An alternative view of Diffusion comes from Score-based modeling. Instead of probability densities, we learn the Score Function: $s(x) = \nabla_x \log p(x)$
This vector field points toward the high-density regions of the data distribution (the "peaks").
Denoising is essentially the act of moving a noisy sample back toward the high-probability manifold.

Score Function

$$s(x) = \nabla_x \log p(x)$$
$$\in \mathbb{R}^d$$

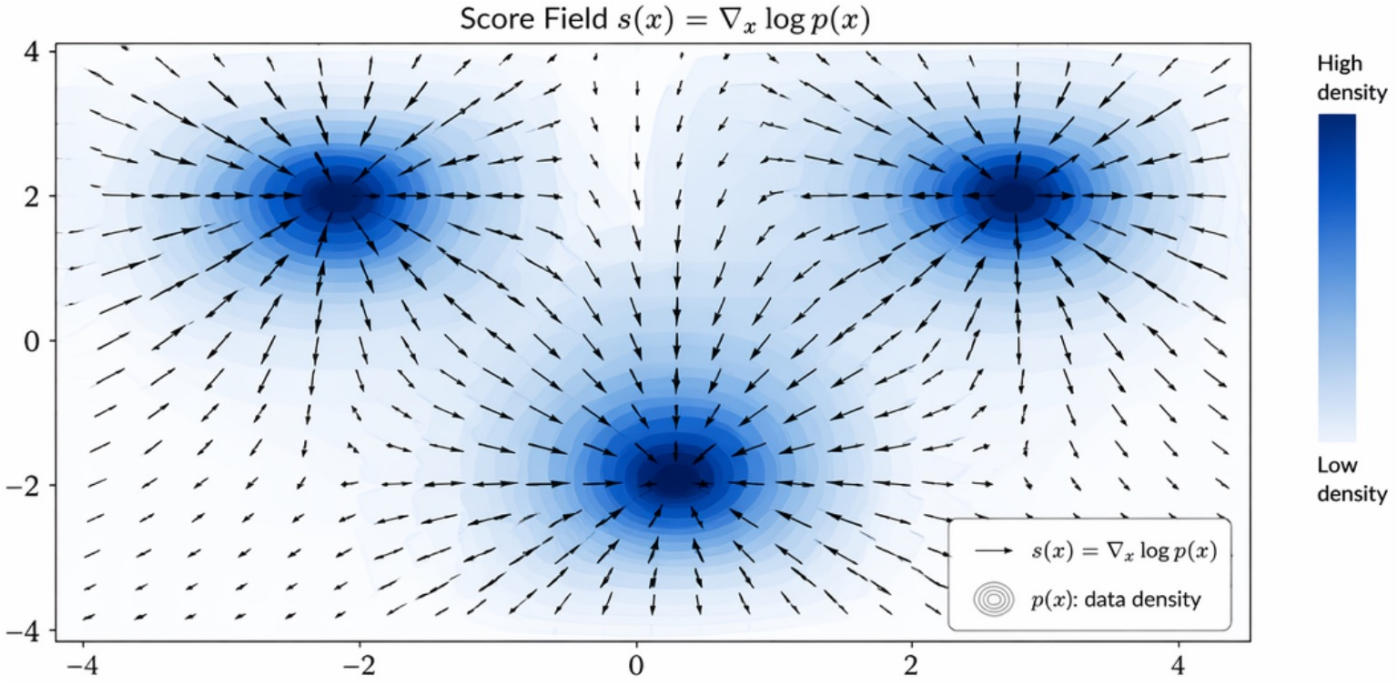
Interpretation

$s(x)$ points in the direction of steepest increase of log-density.

Arrows point toward the high-density regions (modes).

Denoising as Gradient Ascent

Starting from noise, we iteratively move samples in the direction of $s(x)$ to reach the data manifold.

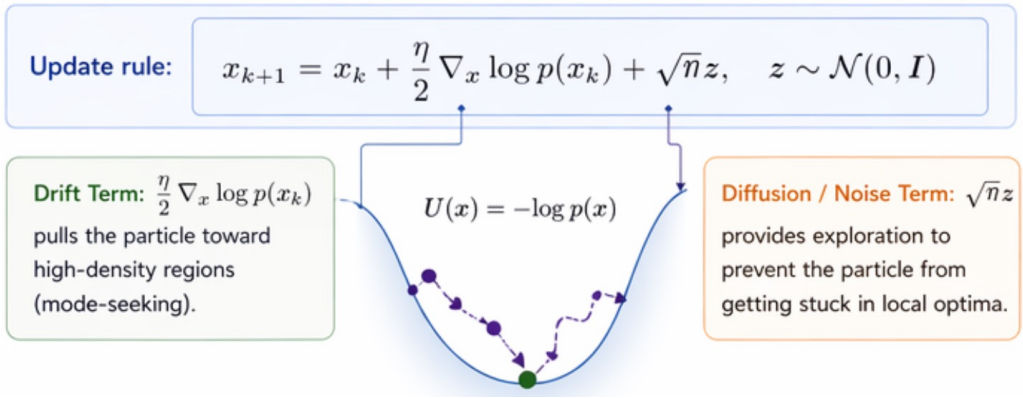


Song, Yang, and Stefano Ermon. "Generative modeling by estimating gradients of the data distribution." NIPS 2019

Score-Based Perspective: The Score Function

Langevin Dynamics describes the trajectory of a particle *x* moving in a potential field *U(x) = -log p(x)* towards the point with the lowest potential, with stochastic thermal fluctuations.

Equivalent to maximize the *p(x),*
which can be the real data distribution,
thus generating realistic data.



In practice, we do not know *p(x)*.

Train a neural network to approximate the score function:

$$s_\theta(x) \approx \nabla_x \log p(x) \quad \Rightarrow \quad \mathcal{L}_{score} = \mathbb{E}_{p(x)}\left[\|s_\theta(x) - \nabla_x \log p(x)\|^2\right]$$

Score-Based Perspective: The Score Function

To solve the "unknown score" issue, we perturb the data x_0 with noise to create x_t. The log-gradient of the conditional distribution is easy to calculate:

$$\log q(x_t|x_0) = -\frac{1}{2\sigma^2}\|x_t - x_0\|^2 + \text{const} \quad \Longrightarrow \quad \nabla_{x_t} \log q(x_t|x_0) = -\frac{x_t - x_0}{\sigma^2}$$

The gradient of the conditional density provides an unbiased estimate of the gradient of the marginal density.

Denoising Score Matching: $\mathbb{E}_{q(x_0,x_t)}[\|s_\theta(x_t) - \nabla_{x_t} \log q(x_t|x_0)\|^2] = \mathbb{E}_{p(x_t)}[\|s_\theta(x_t) - \nabla_{x_t} \log p(x_t)\|^2]$

1. We know $\nabla_{x_t} \log q(x_t|x_0) = -\frac{x_t - x_0}{\sigma^2}$.
2. From the noise equation $x_t = x_0 + \sigma\epsilon$, we have $(x_t - x_0) = \sigma\epsilon$.
3. Substituting this: $\nabla_{x_t} \log q(x_t|x_0) = -\frac{\sigma\epsilon}{\sigma^2} = -\frac{\epsilon}{\sigma}$.

$$\mathcal{L}_{DSM} = \mathbb{E}_{x_0,\epsilon}\left[\left\|s_\theta(x_t) - \left(-\frac{\epsilon}{\sigma}\right)\right\|^2\right]$$

Deriving

In score-based modeling and diffusion processes, $x_t = x_0 + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$

This implies that the conditional distribution: $q(x_t|x_0) = \frac{1}{\sqrt{(2\pi\sigma^2)^d}} \exp\left(-\frac{\|x_t - x_0\|^2}{2\sigma^2}\right)$

So: $\log q(x_t|x_0) = -\frac{1}{2\sigma^2}\|x_t - x_0\|^2 + \text{const}$

The first term is a constant with respect to x_t $\log q(x_t|x_0) = \log\left(\frac{1}{\sqrt{(2\pi\sigma^2)^d}}\right) - \frac{\|x_t - x_0\|^2}{2\sigma^2}$

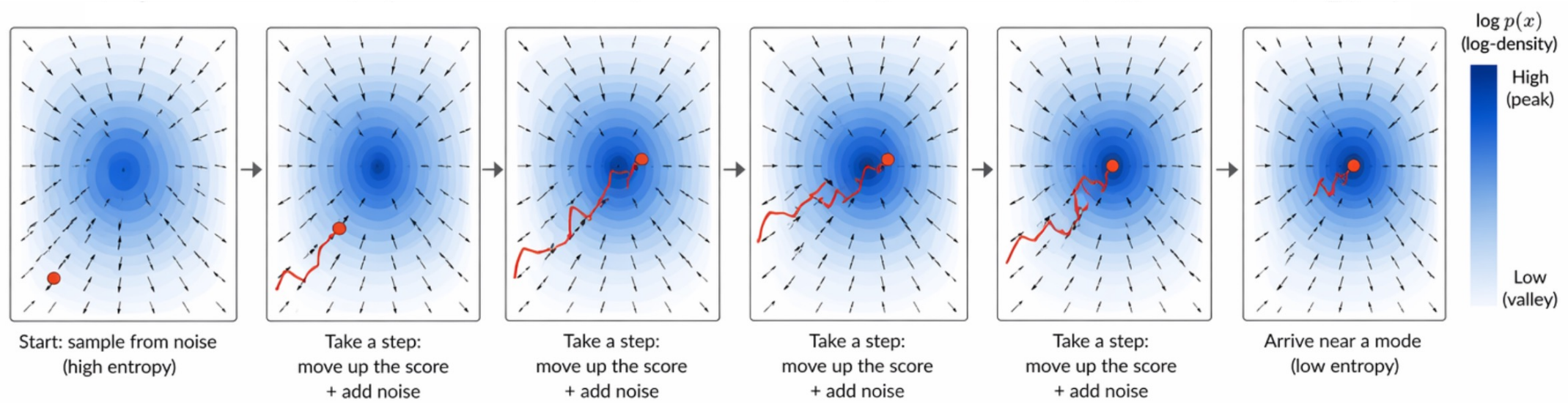
Obtaining the score function: $\nabla_{x_t} \log q(x_t|x_0) = -\frac{x_t - x_0}{\sigma^2}$

Unbiasedness, refer to [Vincent, P.]

Vincent, P. (2011). A Connection Between Score Matching and Denoising Autoencoders. Neural Computation.

Langevin Dynamics for inference (reverse process)

Starting from a random location x_T, use the **trained** score function $s_\theta(x_t)$ to iteratively move x_T to $\hat{x}_0$



Unified Framework: Denoising vs. Score Matching

$$\epsilon_\theta(x_t, t) \;\propto\; -\sigma_t \nabla_x \log p_t(x)$$

Aspect	Diffusion Models (DDPM View)	Score Matching (Score-Based View)
1. Sampling / Generation	Reverse diffusion (DDPM): $x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z,\ z \sim \mathcal{N}(0, I)$ Typical choice: $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)$	Langevin Dynamics (Euler–Maruyama): $x_{t-1} = x_t + \frac{\eta_t}{2} s_\theta(x_t, t) + \sqrt{\eta_t}\, z,\ z \sim \mathcal{N}(0, I)$
2. Dynamics Type	Discrete-time Markov chain (finite T steps)	Continuous-time SDE (infinity small step limit)
3. Key Connection	$\epsilon_\theta(x_t, t) = -\sigma_t \nabla_x \log p_t(x_t) +$ (constant factor)	$\epsilon_\theta(x_t, t) = -\sigma_t \nabla_x \log p_t(x_t) +$ (constant factor)
4. Takeaway	Learn to denoise noisy data and recover clean samples.	Estimate the score function to sample from the data distribution.

★ **Big Idea:** Both methods operate on the same sequence of noisy data distributions $p_t(x)$ and are equivalent up to a constant factor.

Why Diffusion for NLP?

Autoregressive (GPT-style) Generation

Task (Global Constraint):
Write a 1-line poem where the last word is "Sun".

Challenge:
The model only sees the past.
It cannot easily plan for the future.

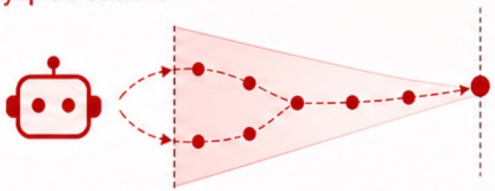
Left-to-Right Generation (Step-by-Step)

Step 1	Step 2	Step 3	Step 4	...	Final Step	
In (high prob)	→ the (high prob)	→ silent (locally likely)	→ midnight (locally likely)	→ ... →	moon (likely)	≠ Sun

By the time it gets to the end, the context is already fixed.
It's hard (or impossible) to end with "Sun" without awkwardness.

The Problem: Myopic Decisions

At each step, GPT chooses the best local word...



...but gets stuck in a corner of the space that doesn't allow the constraint.

Result:
Autoregressive models struggle with global constraints, multi-point attributes, and long-range coherence.

Diffusion Model Generation (Whole-Sequence Refinement)

Same Task (Global Constraint):
Write a 1-line poem where the last word is "Sun".

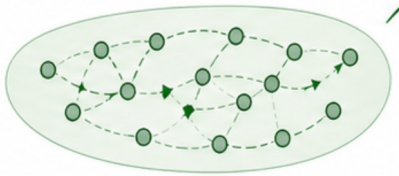
Advantage:
The model sees and refines the entire sequence at once.

Simultaneous Refinement (Many Steps)

Random noise sequence → Structure emerges globally → All tokens aligned; constraint satisfied ✓

Why It Works: Global View Enables Planning

The model optimizes the entire sequence at every step.



It can "plan" for the future, ensuring the last word is "Sun" while keeping the sentence natural.

→ Global alignment ✓

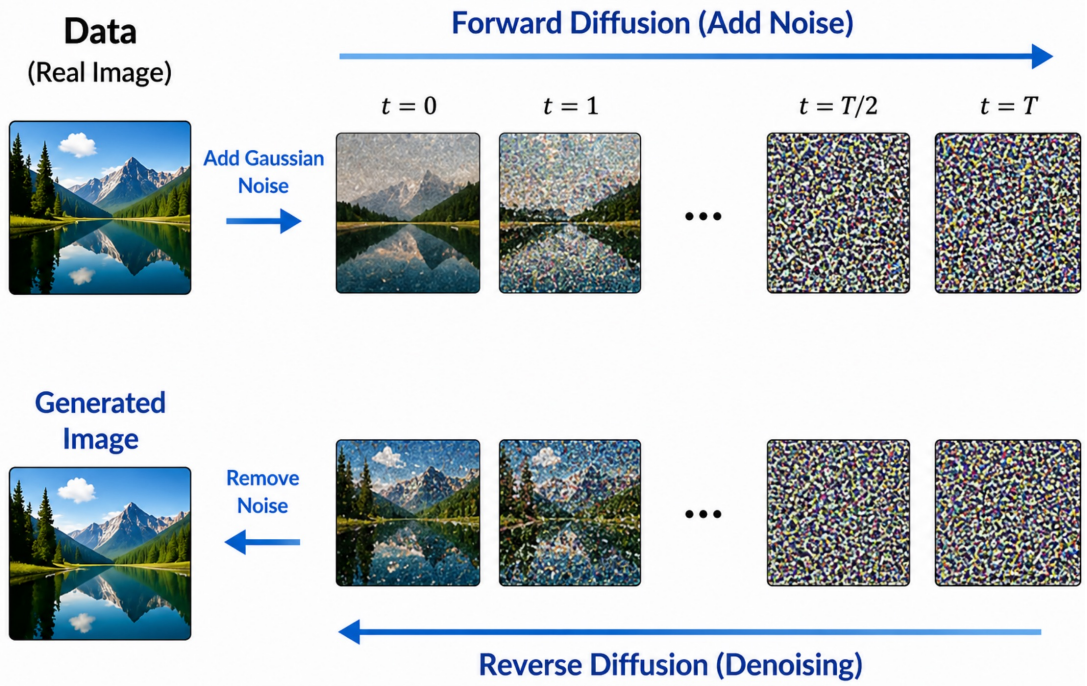
Examples of Control

- ✓ Last word / first word
- ✓ Include specific words or phrases
- ✓ Follow a rhyme or meter scheme
- ✓ Match a topic and sentiment
- ✓ Avoid certain words
- ✓ Enforce entity relationships
- ✓ And many more!

Result:
Diffusion models naturally excel at global constraints, multi-point control, style blending, and long-range coherence.

Advanced Diffusion: From Images to Text

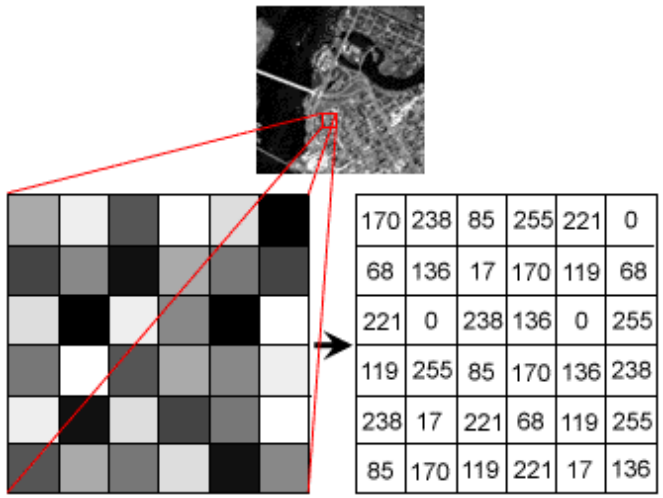
Treating with image pixels as continuous variables are easy: the distance metric and ordering are well-defined.



sentence=[w_1, w_2 ,w_3], $w_i \in V, i = 1,2,3$

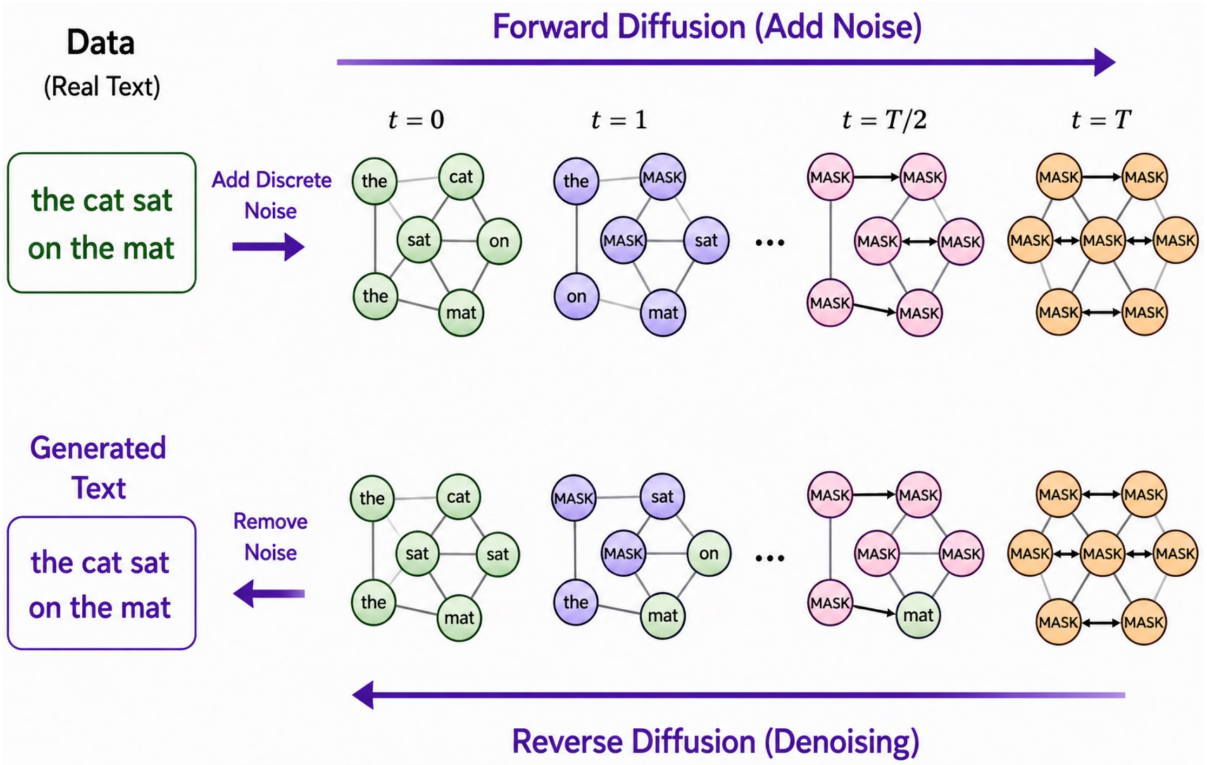
There is no order or distance metric defined on V

But languages, on the other hand, are not too much different from images.



These are discrete values too.

Diffusion of text samples



Need to be careful about text perturbations

Key Idea

In NLP, tokens are discrete indices.
Adding Gaussian noise to a word embedding moves the point into **"void"** space, where it may not correspond to any valid word.

1 Start: A valid word

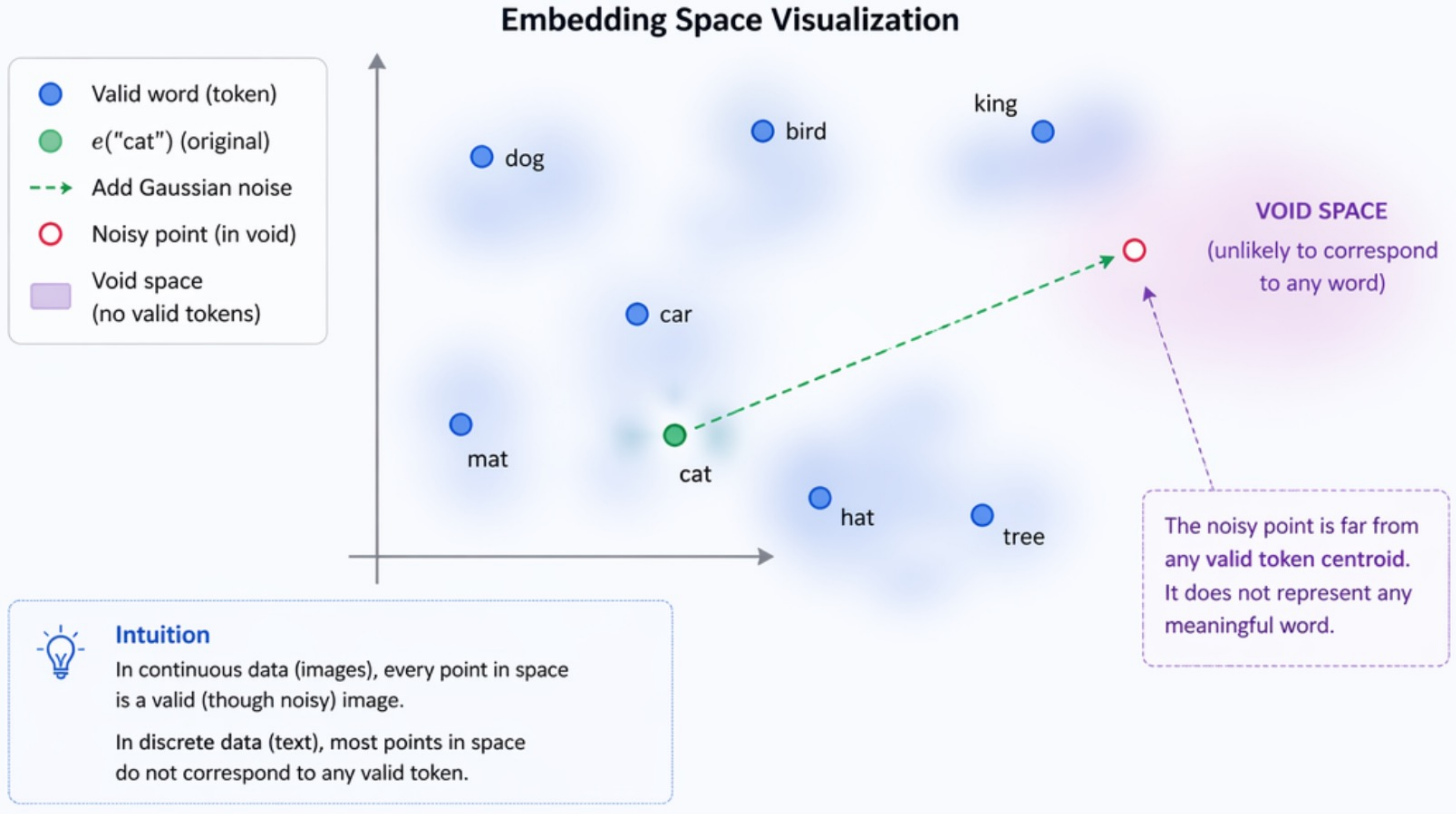
The word "cat" has embedding $e(\text{"cat"})$ in the embedding space.

2 Add Gaussian noise

We add noise $\sim \mathcal{N}(0, \sigma^2 \mathrm{I})$ to the embedding.

3 Lands in "void" space

The noisy point is likely far from any valid token (no meaning).



From Gaussian Kernels to Transition Matrices

To define diffusion over a discrete vocabulary *V*, we replace the Gaussian kernel with a Transition Matrix $Q_t \in \mathbb{R}^{V \times V}$.

Instead of shifting a mean, we define the probability of a token x_{t-1} "jumping" to another token x_t. The forward process is now a categorical Markov chain:

$x_t = Q_t \, x_{t-1}$: transiting from a probability distribution x_{t-1} over tokens to another distribution x_t.

Examples:

How to Forward Step Works

Given current token x_{t-1} with one-hot vector $\boldsymbol{x_{t-1}}$
Three kinds of transitions:

Keep:

The token remains unchanged at its current position.
In the early stages of the forward process (close to $t=0$), most tokens are "kept" to maintain the original data structure.
Represents the probability $1-\beta_t$ that a token survives the transition step without being corrupted.

$$Q_t = (1-\beta_t)\boldsymbol{I} + \beta_t U$$

I : identity matrix (keep the same token)
U : uniform matrix (all entries $= 1/V$)
$\beta_t \in [0, 1]$: noise level at step t

Mask:

The token is replaced by a special [MASK] token, which is unknown to the model.
This is the most popular form of discrete diffusion, often referred to as "absorbing" because the [MASK] state acts as a sink.

$$Q_t = (1-\beta_t)\boldsymbol{I} + \beta_t \boldsymbol{e}_{mask}\,\mathbf{1}^T$$

- I : identity matrix (keep the same token)
- $\boldsymbol{e}_{mask}$: one-hot vector for [MASK] token
- $\mathbf{1}$: all-ones column vector
- $\beta_t \in [0, 1]$: noise level at step t

Swap:

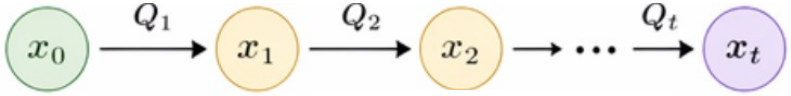
The original token is replaced by another token chosen from the vocabulary, either uniformly or according to a pre-defined probability matrix.
This introduces noise by replacing valid data with random incorrect data, sometimes referred to as "uniform diffusion".

$$Q_t = (1-\beta_t)\boldsymbol{I} + \beta_t \frac{1}{V-1}(\mathbf{1}\mathbf{1}^\top - \boldsymbol{I})$$

- I : identity matrix (keep the same token)
- $\mathbf{1}$: all-ones column vector
- V : vocabulary size
- $\beta_t \in [0, 1]$: noise level at step t

The Forward Posterior in Discrete Space

1. Forward Process in Discrete Space:



2. We define: $\bar{\mathbf{Q}}_t = \mathbf{Q}_1 \mathbf{Q}_2 \dots \mathbf{Q}_t$.

3. Forward Posterior via Bayes' Rule:

$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

4. In terms of transition matrices:

$$q(x_t \mid x_{t-1}) = Q_t(x_{t-1}, x_t)$$

$$q(x_{t-1} \mid x_0) = \bar{Q}_{t-1}(x_0, x_{t-1})$$

$$q(x_t \mid x_0) = \bar{Q}_t(x_0, x_t)$$

$$q(x_{t-1} \mid x_t, x_0) = \frac{Q_t(x_{t-1}, x_t)\ \bar{Q}_{t-1}(x_0, x_{t-1})}{\bar{Q}_t(x_0, x_t)}$$

5. Training the Reverse Model:

We train a model to approximate the true reverse posterior $q(x_{t-1}|x_t, x_0)$

Objective: Minimize KL Divergence

$$\mathcal{L}_{\text{KL}} = \mathbb{E}_{q(x_0)\,q(x_t \mid x_0)}\Big[D_{\text{KL}}\left(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t)\right)\Big]$$

based on variational lower bound from the last lecture.

For discrete distribution, this KL is minimized by the Category Cross-Entropy loss:

Categorical Cross-Entropy Loss

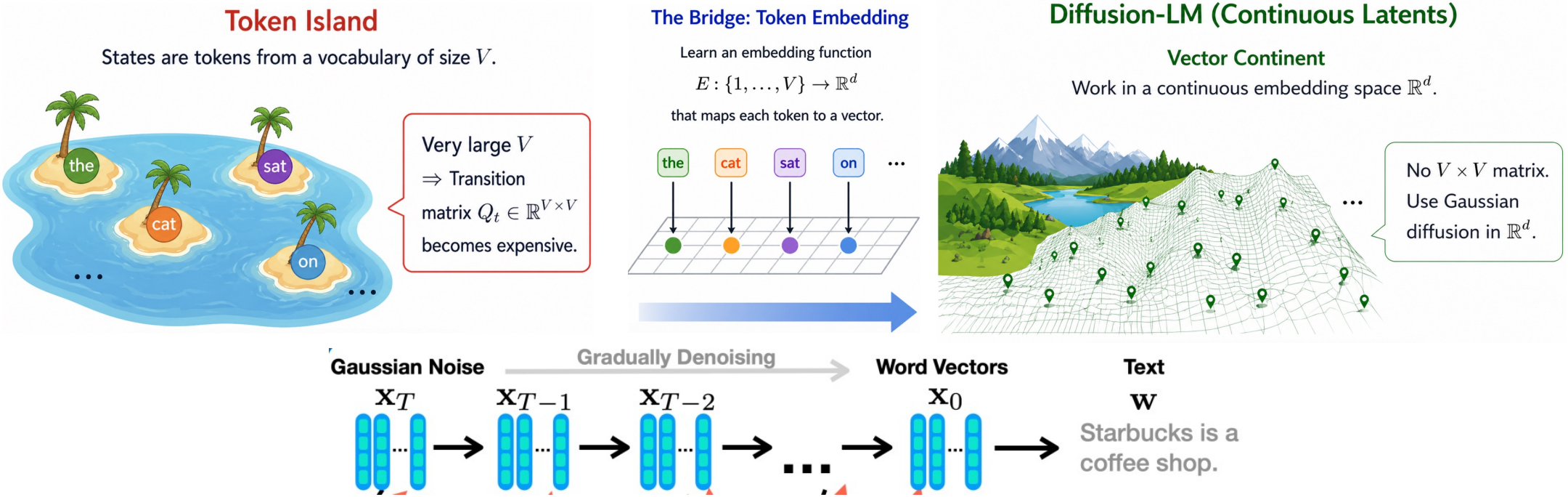
$$\mathcal{L}_{\text{CE}} = -\,\mathbb{E}_{q(x_0)\,q(x_t \mid x_0)}\Big[\log p_\theta(x_{t-1} = \hat{x}_{t-1} \mid x_t)\Big]$$

where $\hat{x}_{t-1} \sim q(x_{t-1} \mid x_t, x_0)$

6. We can train a categorical predictor to reverse the corruption process.

Diffusion in Continuous Latent Spaces

When the vocabulary is large, discrete diffusion becomes computationally expensive (matrices are *V x V*). Diffusion-LM takes a different path: map tokens to a continuous embedding space (**with order and metric!**) and apply Gaussian Diffusion.

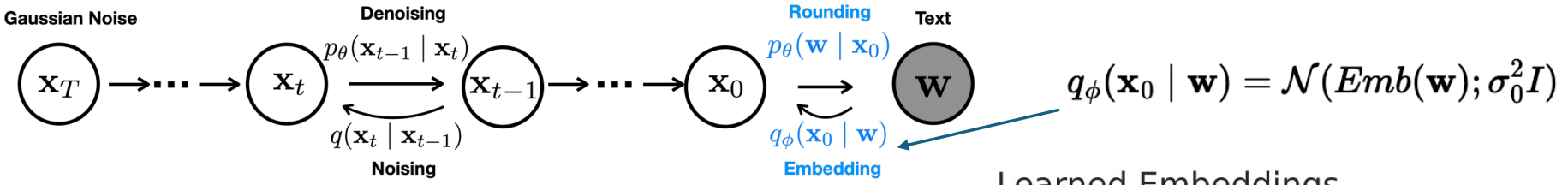


Diffusion-LM iteratively denoises a sequence of Gaussian vectors into word vectors, yielding a intermediate latent variables of decreasing noise level $\{X_T, \ldots, X_0\}$

Li, Xiang, et al. "Diffusion-lm improves controllable text generation." NIPS (2022): 4328-4343.

Solution

Suppose we have a sequence of words: $\mathbf{w} = \{w_1, w_2, \ldots, w_n\}$. An embedding function maps each word into a vector: $Emb(w_i) \in \mathbb{R}^d$.
Thus, the entire sequence is encoded into: $\mathbf{x}_0 = Emb(\mathbf{w}) \in \mathbb{R}^{n \times d}$



Loss function:

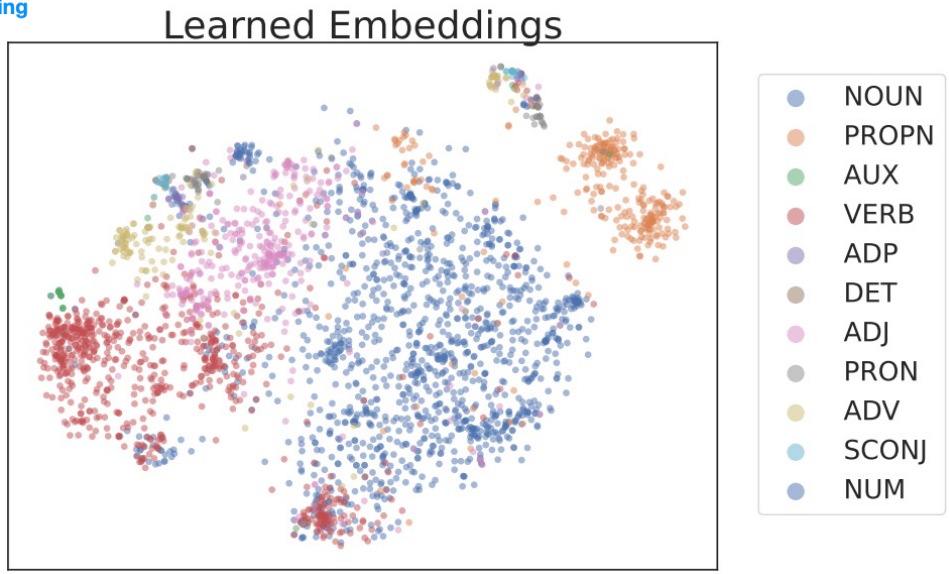
$$\mathcal{L}^{e2e}_{simple}(\mathbf{w}, \theta) = \mathop{\mathbb{E}}_{q_\phi(\mathbf{x}_{0:T}|\mathbf{w})} \left[\underbrace{\mathcal{L}_{simple}(\mathbf{x}_0)}_{\text{diffusion Loss}} + ||Emb(\mathbf{w}) - \overbrace{\mu_\theta(\mathbf{x}_1, 1)}^{\text{predicted } \mathbf{x}_0}||^2 - \underbrace{\log p_\theta(\mathbf{w}|\mathbf{x}_0)}_{\text{rounding}} \right]$$

diffusion Loss:

$$\mathcal{L}_{simple}(\mathbf{x}_0, \theta) = \sum_{t=1}^{T} \mathop{\mathbb{E}}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[\|\mu_\theta(\mathbf{x}_t, t) - \hat{\mu}(\mathbf{x}_t, \mathbf{x}_0)\|^2 \right],$$

] rounding to discrete tokens using softmax

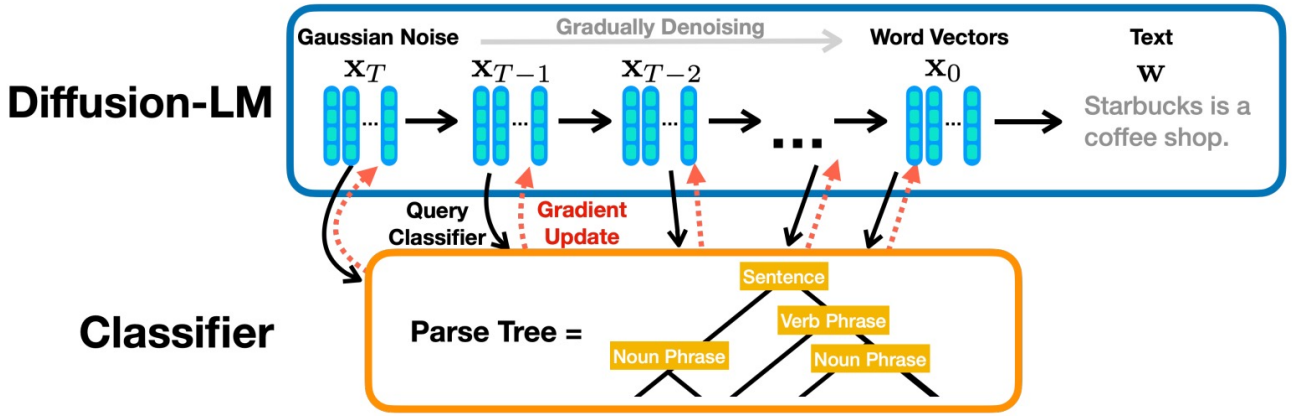
$$p_\theta(\mathbf{w} \mid \mathbf{x}_0) = \prod_{i=1}^{n} p_\theta(w_i \mid x_i)$$



words with the same part-of-speech tags (syntactic role) tend to be clustered

How to control our text generation?

We can now generate the correct text, but how do we generate the text we want?



We iteratively perform gradient updates on these continuous embeddings to optimize for fluency and satisfy control requirements *c* (parametrized by a classifier).

Controlling $\mathbf{x}_{0:T}$ is equivalent to decoding from the posterior $p(\mathbf{x}_{0:T}|\mathbf{c}) = \prod_{t=1}^{T} p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{c})$ and we decompose this joint inference problem to a sequence of control problems at each diffusion step:

$$p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{c}) \propto p(\mathbf{x}_{t-1} \mid \mathbf{x}_t) \cdot p(\mathbf{c} \mid \mathbf{x}_{t-1}, \mathbf{x}_t) \qquad \text{(Bayes Theorem)}$$

$$\nabla_{\mathbf{x}_{t-1}} \log p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{c}) = \nabla_{\mathbf{x}_{t-1}} \log p(\mathbf{x}_{t-1} \mid \mathbf{x}_t) + \nabla_{\mathbf{x}_{t-1}} \log p(\mathbf{c} \mid \mathbf{x}_{t-1}),$$

Deriving

Controlling $\mathbf{x}_{0:T}$ is equivalent to decoding from the posterior $p(\mathbf{x}_{0:T}|\mathbf{c}) = \prod_{t=1}^{T} p(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{c})$ and we decompose this joint inference problem to a sequence of control problems at each diffusion step:

$$p(x_{t-1}|x_t, c) \propto p(x_{t-1}|x_t) \cdot p(c|x_{t-1}, x_t)$$

To simplify the second term, we adopt conditional independence assumptions from prior work on controlling diffusions:

$$p(c|x_{t-1}, x_t) \approx p(c|x_{t-1})$$

To perform gradient-based optimization, we take the logarithm of both sides:

$$\log p(x_{t-1}|x_t, c) = \log p(x_{t-1}|x_t) + \log p(c|x_{t-1}) + \text{const}$$

So:

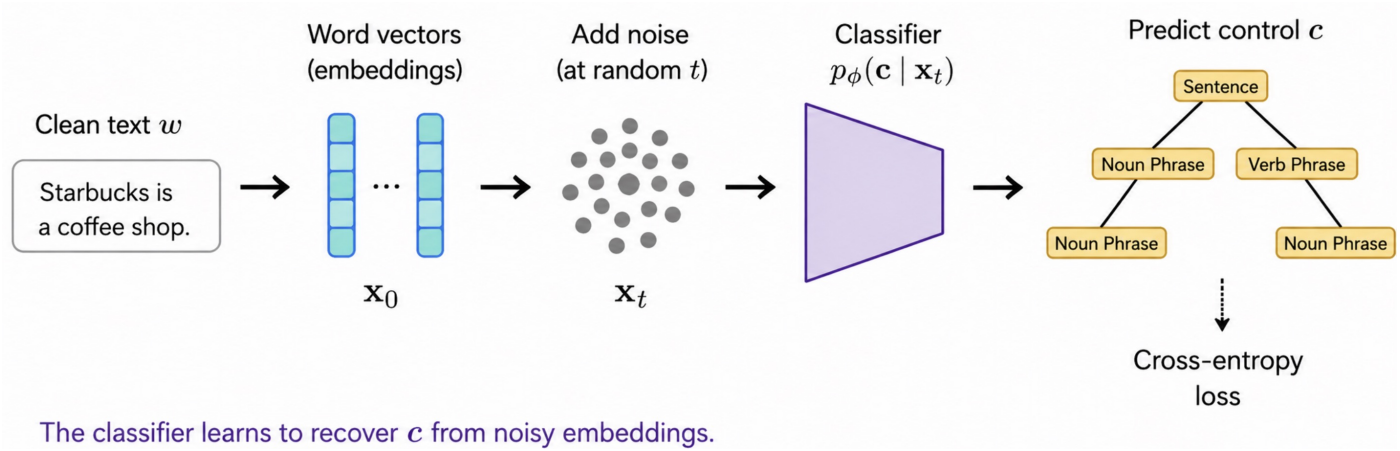
$$\nabla_{x_{t-1}} \log p(x_{t-1}|x_t, c) = \nabla_{x_{t-1}} \log p(x_{t-1}|x_t) + \nabla_{x_{t-1}} \log p(c|x_{t-1})$$

How to train a classifier

A labeled dataset $\mathcal{D} = \{(w, c)\}$, consisting of text sequences w and their corresponding control labels c.
Classifier ϕ

Repeat until convergence:

1. Sample $(w, c) \sim \mathcal{D}$.
2. Map text to clean embedding: $x_0 \sim q_\phi(x_0|w) = \mathcal{N}(EMB(w), \sigma_0 I)$.
3. Sample a random timestep: $t \sim \text{Uniform}(\{1, \ldots, T\})$.
4. Generate noisy latent via forward process: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$.
5. Predict control attribute: $\hat{p} = \text{Classifier}_\phi(x_t, t)$.
6. Compute loss: $\mathcal{L} = -\log \hat{p}(c)$.
7. Update **only** classifier parameters ϕ via gradient descent.

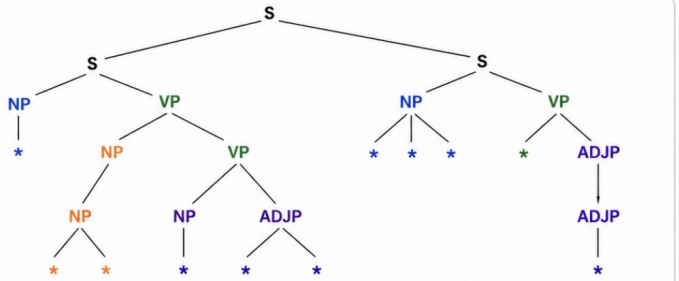


Experiment

Example 1

Target Syntactic Parse (linearized)

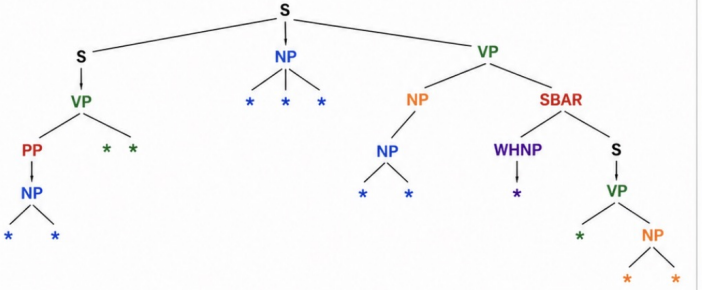
(S (S (NP *) (VP * (NP (NP * *) (VP * (NP (ADJP * *) *)))))
* (S (NP * * *) (VP *) (ADJP (ADJP *))))))



Example 2

Target Syntactic Parse (linearized)

(S (S (VP * (PP * (NP * *))))) * (NP * * *) (VP * (NP (NP * *)
(SBAR (WHNP *) (S (VP * (NP * *))))))) *)



S: Sentence NP: Noun Phrase VP: Verb Phrase PP: Prepositional Phrase
ADJP: Adjective Phrase SBAR: Subordinate Clause WHNP: Wh-Noun Phrase *: Terminal (word)

1. This represents two sentences (clauses) joined by a single connector word (*). The first clause requires a single-word subject (NP *) and a deeply nested verb phrase. The second clause requires a subject that is precisely three words long (NP * * *) followed by a verb and an adjective phrase.

2. The notation includes an SBAR, which requires the model to generate a relative clause.

Syntactic Parse	(S (S (NP *) (VP * (NP (NP * *) (VP * (**NP (ADJP * *) *)**))))) * (S (NP * * *) (VP * (ADJP (ADJP *)))))
FUDGE	Zizzi is a cheap restaurant . **[incomplete]**
Diffusion-LM	Zizzi is a pub providing **family friendly Indian food** Its customer rating is low
FT	Cocum is a Pub serving **moderately priced meals** and the customer rating is high
Syntactic Parse	(S (S (VP * (PP * (NP * *)))) * **(NP * * *)** (VP * (NP (NP * *) (SBAR (WHNP *) (S (VP * (NP * *)))))) *)
FUDGE	In the city near The Portland Arms is a coffee and fast food place named The Cricketers which is not family - friendly with a customer rating of 5 out of 5 .
Diffusion-LM	Located on the riverside , **The Rice Boat** is a restaurant that serves Indian food .
FT	Located near The Sorrento, **The Mill** is a pub that serves Indian cuisine.

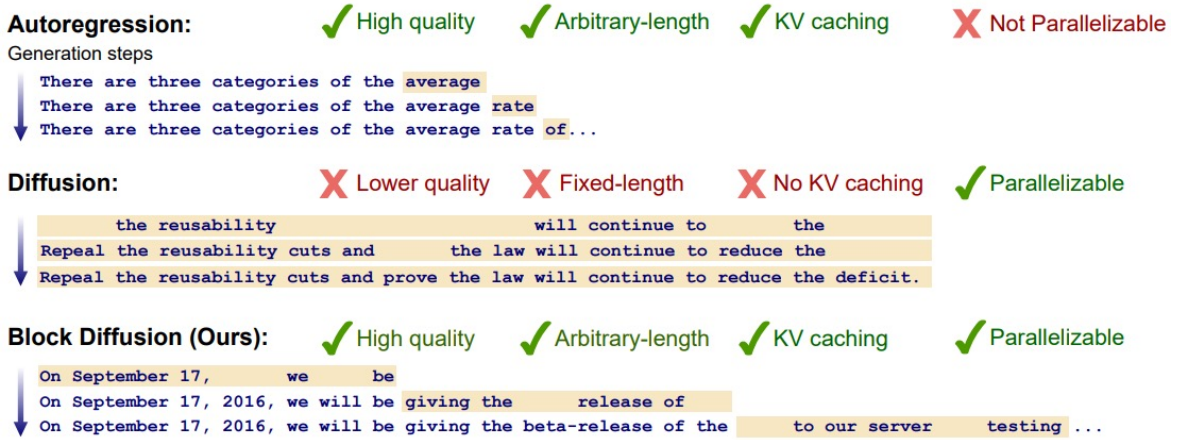
Table 3: Qualitative examples from the Syntax Tree control. The syntactic parse tree is linearized by nested brackets representing the constituents, and we use the standard PTB syntactic categories.

FUDGE: Controlled text generation with future discriminators. ACL 2021
FT Fine-tuning Oracle

S: Sentence NP: Noun Phrase VP: Verb Phrase PP: Prepositional Phrase ADJP: Adjective Phrase SBAR: Subordinate Clause WHNP: Wh-Noun Phrase *: Terminal (word)

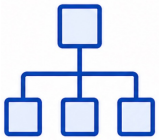
Case Study: Block Diffusion

Discrete diffusion models are limited by fixed-length generation and the inability to reuse computation via KV-caching. Block Diffusion introduces a hybrid paradigm that interpolates between Autoregressive (AR) and Diffusion models.



1 Hierarchical Factorization

The sequence likelihood is factorized over blocks:

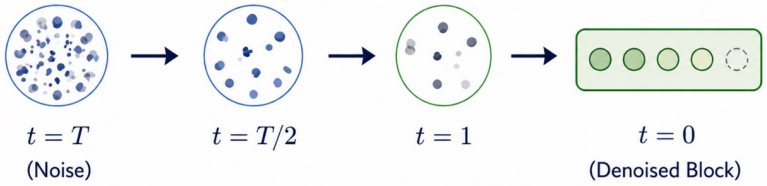


$$p_\theta(\mathbf{x}) = \prod_{b=1}^{B} p_\theta(\mathbf{x}_b \mid \mathbf{x}_{<b})$$

Each block $\mathbf{x}_b$ is generated conditioned on all previous blocks $\mathbf{x}_{<b}$.

2 Intra-block Diffusion

For each block b, we use a discrete denoising diffusion process to sample all tokens in parallel, conditioned on the fixed KV-cache of previous blocks.



Parallel denoising within the block

Arriola, Marianne, et al. "Block diffusion: Interpolating between autoregressive and diffusion language models."ICLR 2025.

Autoregressive vs. Diffusion

Unlike GPT, where a single "bad" word can derail the entire sentence (exposure bias), Diffusion can self-correct. In early steps, it sets the global semantics; in later steps, it fixes local syntax.

Aspect	Autoregressive (GPT-style)	Diffusion (Diffusion-LM)
Factorization	Causal / Left-to-Right $p(x) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$	Non-Causal / Global Model $p(x_T)$ and refine to $p(x_0)$
Generation Speed	Fast (1 forward pass per token)	Slow (Vanilla) (T forward passes, $T \gg$ sequence length)
Information Flow	Unidirectional (Past → Future)	Bidirectional (Global context at every step)
Planning Capability	Weak Global Planning (Hard to enforce future constraints)	Strong Global Planning (Sees the whole sequence during denoising)
Strengths	✓ Extremely fast inference ✓ Excels at short-form, open-ended generation ✓ Huge ecosystem & mature tooling	✓ Superior controllability (style, structure, constraints) ✓ Better coherence for long-range dependencies ✓ Self-correction & iterative refinement
Weaknesses	✗ Exposure bias / Error accumulation ✗ Struggles with global constraints ✗ Degeneration in long sequences	✗ Slow inference (though improving) ✗ Higher computational cost ✗ More complex training & sampling